UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS
FACULTAD DE INGENIERÍA ELECTRÓNICA Y ELÉCTRICA
ESCUELA PROFESIONAL DE INGENIERÍA BIOMÉDICA

INSTRUMENTACIÓN BIOMÉDICA I

INFORME DE LABORATORIO N.° 3

Simulador de sEMG a partir de registros reales

Integrantes

N.°	Apellidos y nombres	Código
1	Lea Yopla Carlos Rodrigo	24190292
2	Mamani Chavez Lizbeth Rocio	24190105
3	Gironzini Córdova Enrico Salvatore	23190373

Grupo / horario	Grupo A: 8 - 10pm
Fecha	28 de septiembre de 2026
Docente	Maria Elisia Armas Alvarado

1. Resumen

En este laboratorio se desarrolló un simulador de señales electromiográficas de superficie (sEMG) utilizando registros reales de la base de datos GRABMyo. Mediante Python en Google Colab, se procesaron y concatenaron tres condiciones musculares: reposo, cierre de mano y extensión del índice. La secuencia de 30 720 muestras se escaló a 8 bits y se reprodujo mediante el DAC del ESP32 a 2048 Hz. Se evidenció que el escalado global permitió conservar las diferencias de amplitud entre las actividades musculares, obteniéndose una señal útil para la validación de sistemas de adquisición de bioseñales.

2. Objetivos

- Leer en Google Colab tres registros de sEMG de la base GRABMyo (reposo, cierre de mano y extensión del índice) de un mismo canal del antebrazo.
- Construir con esos registros una secuencia reposo – cierre de mano – extensión del índice, escalada al rango del DAC de 8 bits y exportada como arreglo para el microcontrolador.
- Reproducir la secuencia por el DAC de un ESP32 a la frecuencia de muestreo original.

3. Marco teórico y contexto de aplicación

Electromiografía de superficie (sEMG)

La electromiografía de superficie (sEMG) es una técnica no invasiva que permite registrar la actividad eléctrica generada por los músculos esqueléticos mediante electrodos colocados sobre la piel. Estas señales presentan un comportamiento variable y de naturaleza aleatoria, con amplitudes que generalmente oscilan entre decenas de microvoltios y algunos milivoltios, y un contenido frecuencial útil aproximado de 20 a 500 Hz. Su amplitud depende de factores como la activación muscular, el reclutamiento de unidades motoras y la ubicación de los electrodos.

En la experiencia se utilizaron registros reales de la base de datos GRABMyo, correspondientes a tres condiciones: reposo, cierre de mano y extensión del índice. Como se observa en la primera figura, durante el reposo se presenta una actividad eléctrica reducida, mientras que el cierre de mano genera fluctuaciones de mayor amplitud. Por su parte, la extensión del índice presenta una actividad intermedia, permitiendo identificar las diferencias de activación muscular entre los movimientos.

Conversión digital-analógica y cuantización

La conversión digital-analógica (DAC) consiste en transformar valores digitales discretos en niveles de tensión analógica. En esta experiencia se emplea el convertidor DAC de 8 bits integrado en el ESP32, capaz de representar 256 niveles digitales, comprendidos entre 0 y 255. La cuantización limita la resolución de la señal reproducida, debido a que cada muestra se aproxima a uno de los niveles disponibles.

La relación ideal entre el valor digital y la tensión de salida se expresa mediante:

$$V_{out} = \frac{D}{255} V_{ref}$$

Simuladores de bioseñales en instrumentación biomédica

Los simuladores de bioseñales permiten generar señales eléctricas conocidas y reproducibles para evaluar el funcionamiento de equipos biomédicos sin necesidad de realizar mediciones continuas sobre pacientes. Su aplicación facilita las pruebas de amplificación, adquisición, filtrado y procesamiento de señales fisiológicas.

En esta experiencia, el simulador se construye a partir de registros sEMG reales de GRABMyo, concatenados en una secuencia de 15 segundos y convertidos en valores digitales de 8 bits. Posteriormente, estos datos se reproducen mediante el DAC del ESP32 a una frecuencia de 2048 Hz. De esta manera, se obtiene una señal de prueba repetible que permite evaluar la respuesta de una cadena de acondicionamiento de señales electromiográficas, considerando las limitaciones de cuantización y resolución del convertidor.

4. Desarrollo experimental

Diagrama de bloques

El sistema integra siete funciones consecutivas: selección de registros de GRABMyo, lectura con Python, concatenación temporal, normalización y cuantización, exportación en lenguaje C, reproducción mediante el DAC y medición de la señal analógica.



Figura 1. Diagrama de bloques del circuito simulador sEMG.

Esquemático del montaje

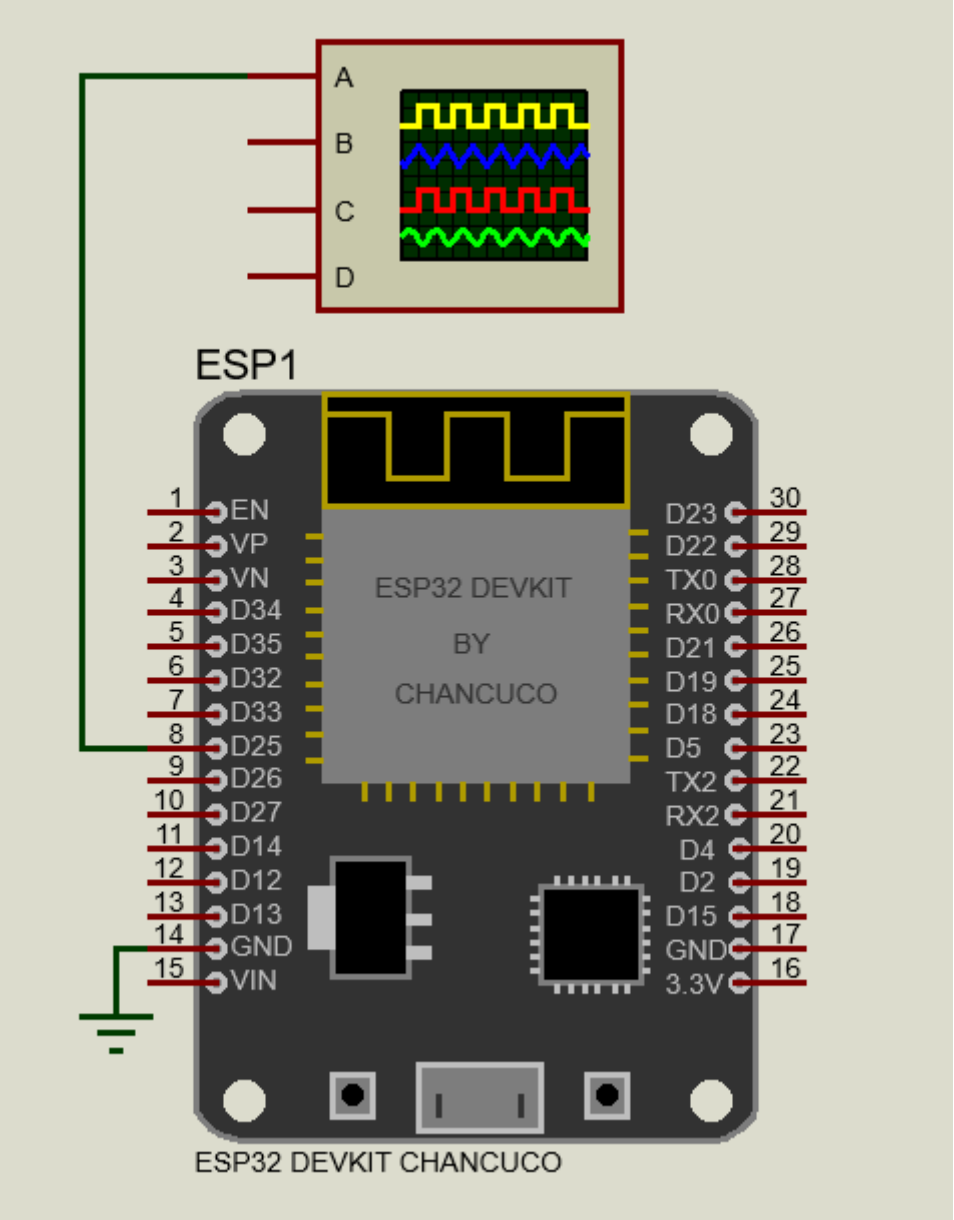


Figura 2. Esquemático eléctrico del ESP32 y del osciloscopio adaptado en Proteus 8 Professional usando la librería de Chancuco.

Fotografía del montaje

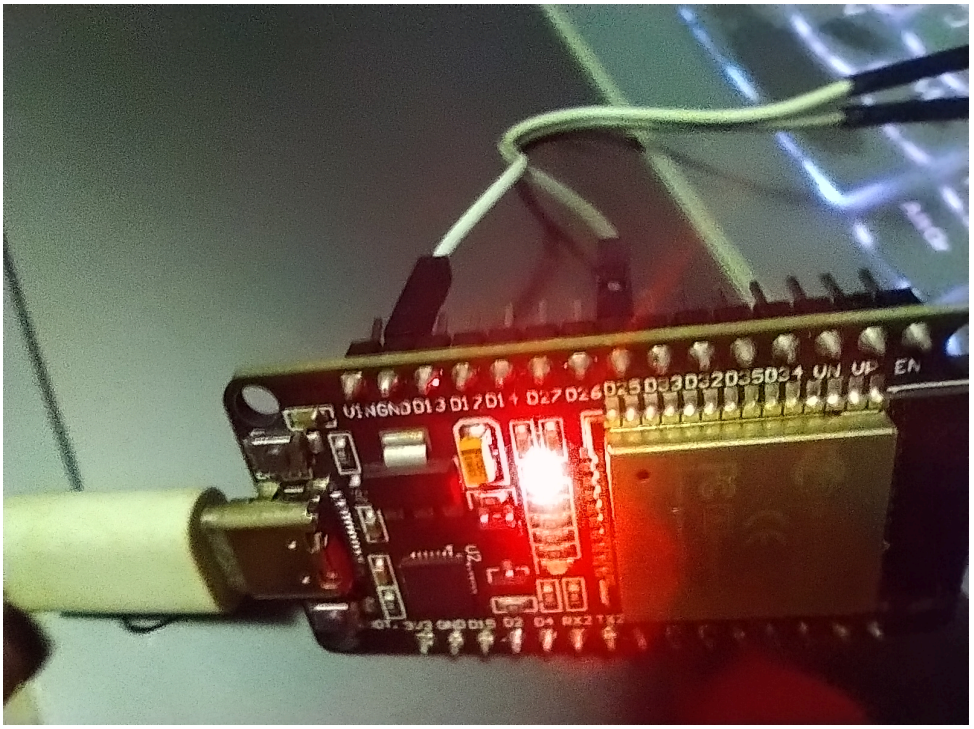


Figura 3: Implementación del circuito simulador de sEMG, pin GPIO 25 conectado a la punta del osciloscopio.

El montaje contempla el uso del ESP32 como generador de la señal analógica, utilizando su convertidor DAC1, disponible en el pin GPIO25. Para la medición, la punta del canal 1 del osciloscopio se conecta a GPIO25, mientras que la pinza de tierra se conecta al GND del ESP32. La placa se alimenta y programa mediante un cable USB conectado a la computadora.

Etapa A — Lectura de registros (Guía, pasos 1 a 3)

Se utilizó Google Colab para acceder a la base de datos GRABMyo mediante la biblioteca wfdb. Se seleccionaron los registros correspondientes al participante 1, sesión 1 y repetición 1, empleando el canal F12, identificado con el índice 11 en Python.

Los registros seleccionados corresponden a reposo (gesto 17), cierre de mano (gesto 16) y extensión del índice (gesto 9).

Se pudo descargar sin inconvenientes los archivos de forma automática.

```
!pip install wfdb

Collecting wfdb
  Downloading wfdb-4.3.1-py3-none-any.whl.metadata (3.8 kB)
Requirement already satisfied: aiohttp>=3.10.11 in /usr/local/lib/python3.13/dist-packages (from wfdb) (3.14.3)
Requirement already satisfied: fsspec>=2023.10.0 in /usr/local/lib/python3.13/dist-packages (from wfdb) (2025.12.0)
Requirement already satisfied: matplotlib>=3.2.2 in /usr/local/lib/python3.13/dist-packages (from wfdb) (3.10.0)
Requirement already satisfied: numpy>=1.26.4 in /usr/local/lib/python3.13/dist-packages (from wfdb) (2.1.3)
Requirement already satisfied: pandas>=2.2.3 in /usr/local/lib/python3.13/dist-packages (from wfdb) (2.2.3)
Requirement already satisfied: requests>=2.8.1 in /usr/local/lib/python3.13/dist-packages (from wfdb) (2.32.4)
Requirement already satisfied: scipy>=1.13.0 in /usr/local/lib/python3.13/dist-packages (from wfdb) (1.16.3)
Requirement already satisfied: soundfile>=0.10.0 in /usr/local/lib/python3.13/dist-packages (from wfdb) (0.14.0)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.13/dist-packages (from aiohttp>=3.10.11->wfdb) (2.7.1)
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.13/dist-packages (from aiohttp>=3.10.11->wfdb) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.13/dist-packages (from aiohttp>=3.10.11->wfdb) (26.1.0)
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.13/dist-packages (from aiohttp>=3.10.11->wfdb) (1.8.0)
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.13/dist-packages (from aiohttp>=3.10.11->wfdb) (6.8.0)
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.13/dist-packages (from aiohttp>=3.10.11->wfdb) (0.5.4)
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.13/dist-packages (from aiohttp>=3.10.11->wfdb) (1.25.1)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=3.2.2->wfdb) (1.4.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=3.2.2->wfdb) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=3.2.2->wfdb) (4.65.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=3.2.2->wfdb) (1.5.1)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=3.2.2->wfdb) (26.3)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=3.2.2->wfdb) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=3.2.2->wfdb) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=3.2.2->wfdb) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages (from pandas>=2.2.3->wfdb) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages (from pandas>=2.2.3->wfdb) (2026.4)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages (from requests>=2.8.1->wfdb) (3.4.9)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist-packages (from requests>=2.8.1->wfdb) (3.19)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13/dist-packages (from requests>=2.8.1->wfdb) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.13/dist-packages (from requests>=2.8.1->wfdb) (2026.7.22)
Requirement already satisfied: cffi>=1.0 in /usr/local/lib/python3.13/dist-packages (from soundfile>=0.10.0->wfdb) (2.1.1)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.13/dist-packages (from soundfile>=0.10.0->wfdb) (4.16.0)
Requirement already satisfied: pycparser in /usr/local/lib/python3.13/dist-packages (from cffi>=1.0->soundfile>=0.10.0->wfdb) (3.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from python-dateutil>=2.7->matplotlib>=3.2.2->wfdb) (1.17.0)
Downloading wfdb-4.3.1-py3-none-any.whl (163 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 163.9/163.9 kB 4.6 MB/s eta 0:00:00
Installing collected packages: wfdb
Successfully installed wfdb-4.3.1
```

Figura 4. Evidencia de la descargar de los archivos de EMG

Posteriormente, se utilizaron NumPy y Matplotlib para organizar y representar las señales en tres gráficas con escalas compartidas, facilitando la comparación de sus amplitudes durante intervalos de cinco segundos.

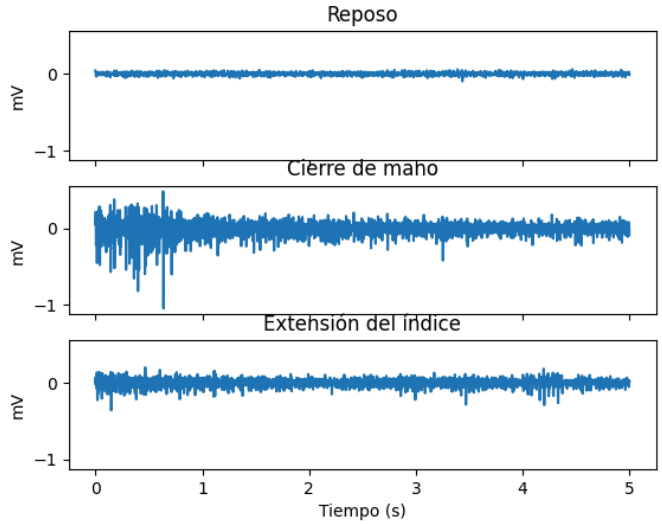


Figura 5. Señal EMG separada por etapas: Reposo, cierre de mano y extensión del índice

Se muestra en la Figura 5 los 3 registros en milivoltios (mV) y cada uno durante 5 segundos cada uno como indica la guía.

Se adjunta además la evidencia del código ejecutado en Colab.

```
import wfdb
import numpy as np
import matplotlib.pyplot as plt
carpeta = "grabmyo/1.1.0/Session1/session1_participant1"
gestos = {"Reposo": 17, "Cierre de mano": 16, "Extensión del índice": 9}
canal = 11 # canal F12 → índice 11 en Python
senales = {}
for nombre_g, k in gestos.items():
 rec = wfdb.rdrecord(f"session1_participant1_gesture{k}_trial1", pn_dir=carpeta)
 senales[nombre_g] = rec.p_signal[:, canal]
fig, ax = plt.subplots(3, 1, sharex=True, sharey=True)
for a, (nombre_g, x) in zip(ax, senales.items()):
 a.plot(np.arange(len(x)) / rec.fs, x)
 a.set_title(nombre_g)
 a.set_ylabel("mV")
ax[-1].set_xlabel("Tiempo (s)")
plt.show()
```

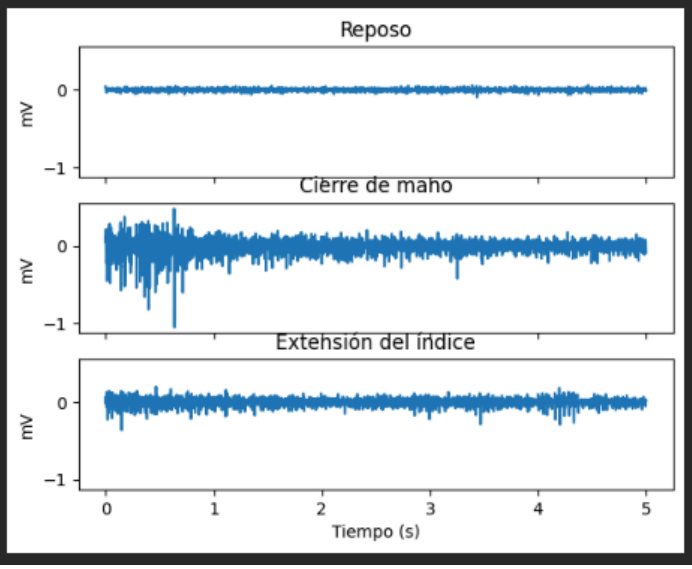


Figura 6. Evidencia de la ejecución del código en Colab

Etapa B — Construcción de la secuencia y exportación (Guía, pasos 4 a 7)

Mediante la función np.concatenate() se unieron los tres registros en el orden reposo, cierre de mano y extensión del índice. En esta etapa se logró concatenar las señales y obtener unas estadísticas importantes que coinciden con la guía como se muestra en la ejecución de código de la Figura 7. hay 30720 muestras, equivalente a 15 segundos con una frecuencia de muestreo de 2048 Hz. desde el valor de 0 hasta 255.

Tabla 1. Resultados obtenidos

Condición	Intervalo	Amplitud pico a pico
Reposo	0–5 s	26 niveles
Cierre de mano	5–10 s	255 niveles
Extensión del índice	10–15 s	94 niveles

```
secuencia = np.concatenate([
    senales["Reposo"],
    senales["Cierre de mano"],
    senales["Extensión del índice"]
])

print("Número de muestras:", len(secuencia))
```

```
Número de muestras: 30720
```

```
minimo = np.min(secuencia)
maximo = np.max(secuencia)

secuencia_escalada = 255 * (secuencia - minimo) / (maximo - minimo)

secuencia_escalada = secuencia_escalada.astype(np.uint8)
```

```
print("Mínimo:", secuencia_escalada.min())
print("Máximo:", secuencia_escalada.max())
print("Número de muestras:", len(secuencia_escalada))
```

```
Mínimo: 0
Máximo: 255
Número de muestras: 30720
```

Figura 7. Estadísticas obtenidas de la ejecución del código en Colab

La Figura 8. muestra a la señal completamente concatenada y con el orden sugerido que comienza con la mano en reposo, después se procede al cierre de mano y finalmente a la extensión del índice.

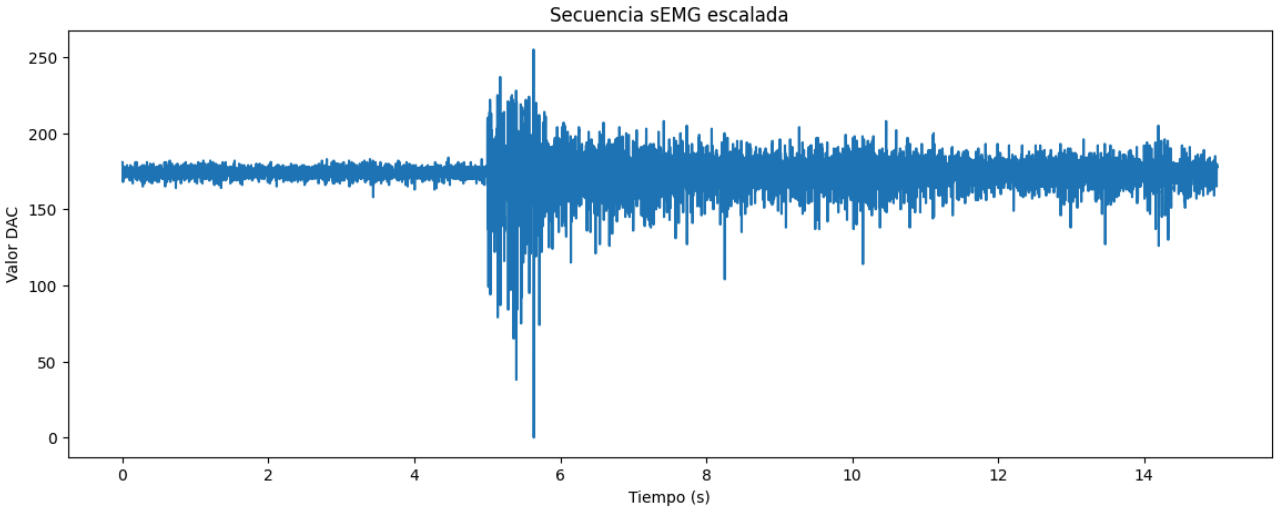


Figura 8. Señal concatenada

Se adjunta además la evidencia del código ejecutado en Colab para obtener la señal completa.

```python
tiempo = np.arange(len(secuencia_escalada)) / 2048

plt.figure(figsize=(14,5))

plt.plot(tiempo, secuencia_escalada)

plt.xlabel("Tiempo (s)")
plt.ylabel("Valor DAC")
plt.title("Secuencia sEMG escalada")

plt.show()
```

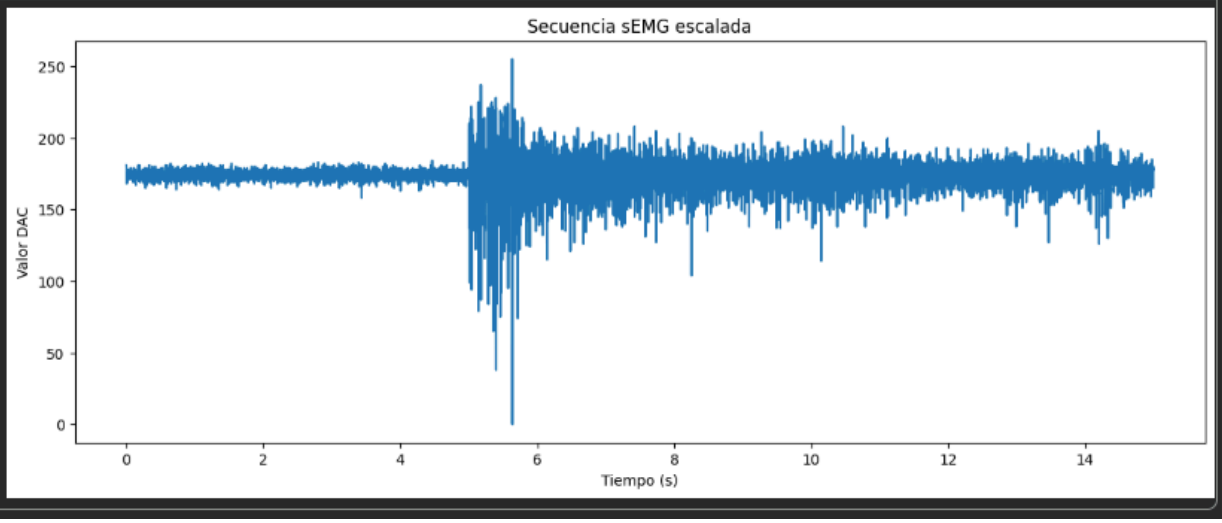


Figura 9. Evidencia de la señal concatenada

Finalmente para esta etapa, se logra crear y descargar el archivo .de cabecera (.h) llamado semg.h para guardar la información de nuestra señal concatenada para usarlo en el código y subirlo a ESP32. Cómo se observa en la figura 10.

```python
with open("semg.h", "w") as f:

    f.write("#include <stdint.h>\n\n")

    f.write("const uint8_t SEMG[] = {\n")

    for i, valor in enumerate(secuencia_escalada):

        f.write(str(valor))

        if i < len(secuencia_escalada) - 1:
            f.write(",")

        if (i + 1) % 20 == 0:
            f.write("\n")

    f.write("\n};\n\n")

    f.write(f"const unsigned int N_SEMG = {len(secuencia_escalada)};\n")

    f.write("const unsigned int FS_SEMG = 2048;\n")
```

```python
from google.colab import files

files.download("semg.h")
```

Figura 10. Evidencia de la creación y descarga de semg.h

Etapa C — Reproducción por el DAC (Guía, pasos 8 a 10)

Se utilizó Arduino IDE para implementar el firmware del ESP32, incorporando el archivo semg.h, que contiene 30 720 muestras de la señal sEMG cuantizadas a 8 bits. Mediante la función dacWrite() se

programó la reproducción de la secuencia por el pin GPIO25. La temporización se realizó utilizando micros(), estableciendo un intervalo programado de 488 µs entre muestras y reiniciando el índice al completar el arreglo para permitir su reproducción continua como se observa en la Figura 11.

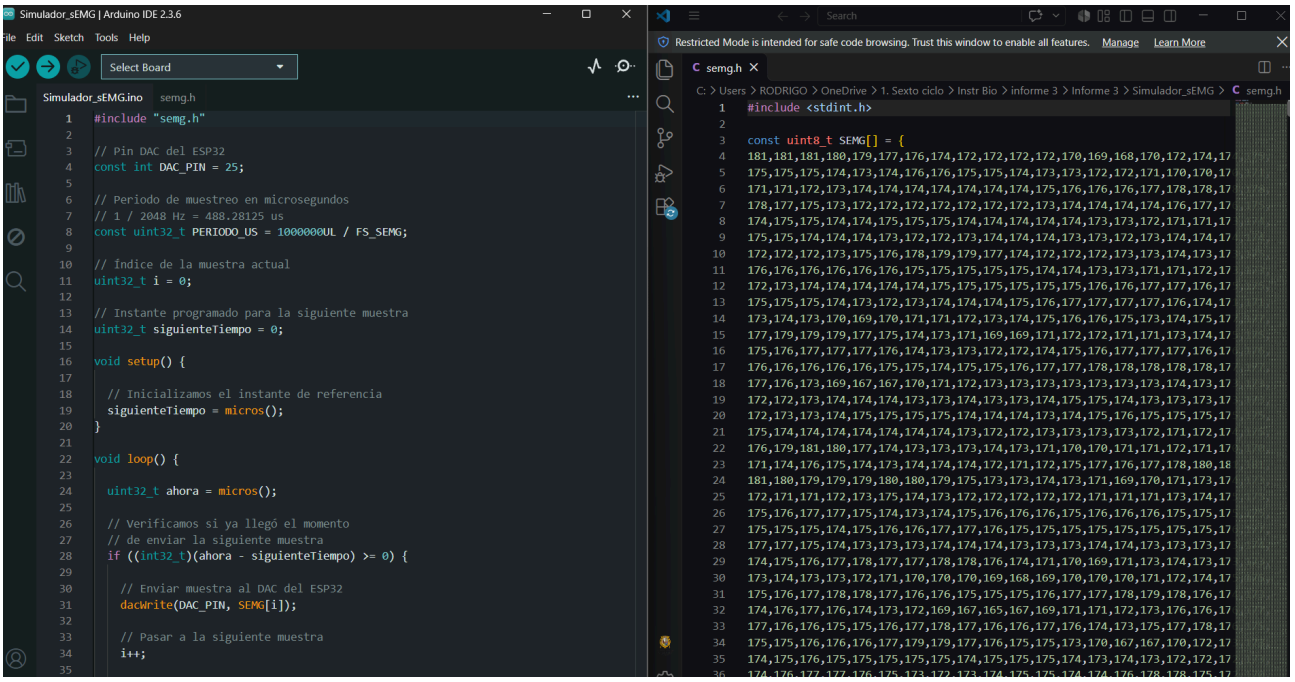


Figura 11. Evidencia del archivo .ino y la existencia del archivo semg.h

Posteriormente, se conectó el canal 1 del osciloscopio al GPIO25 y su referencia de tierra al GND del ESP32. Durante la medición se utilizó una escala horizontal de 1.00 s/div y una escala vertical de 500 mV/div. Se observó la variación de amplitud de la señal y se registraron los valores máximo y pico a pico mediante las funciones de medición automática del osciloscopio.

Tabla 2. Mediciones de la salida del DAC en GPIO25.

Medición	Valor teórico	Valor medido	Observaciones
Periodo de repetición de la secuencia (s)	15	15 s	Coincide con el valor teórico.
Duración de un escalón (ms)	0.488ms	-	Se requiere ampliar la escala temporal y medir con cursores.
Tensión máxima (V)	3.3V	3.17V	Valor cercano al máximo ideal del DAC.
Tensión mínima (V)	0V	40mV	Calculada a partir de la tensión máxima y la amplitud pico a pico.

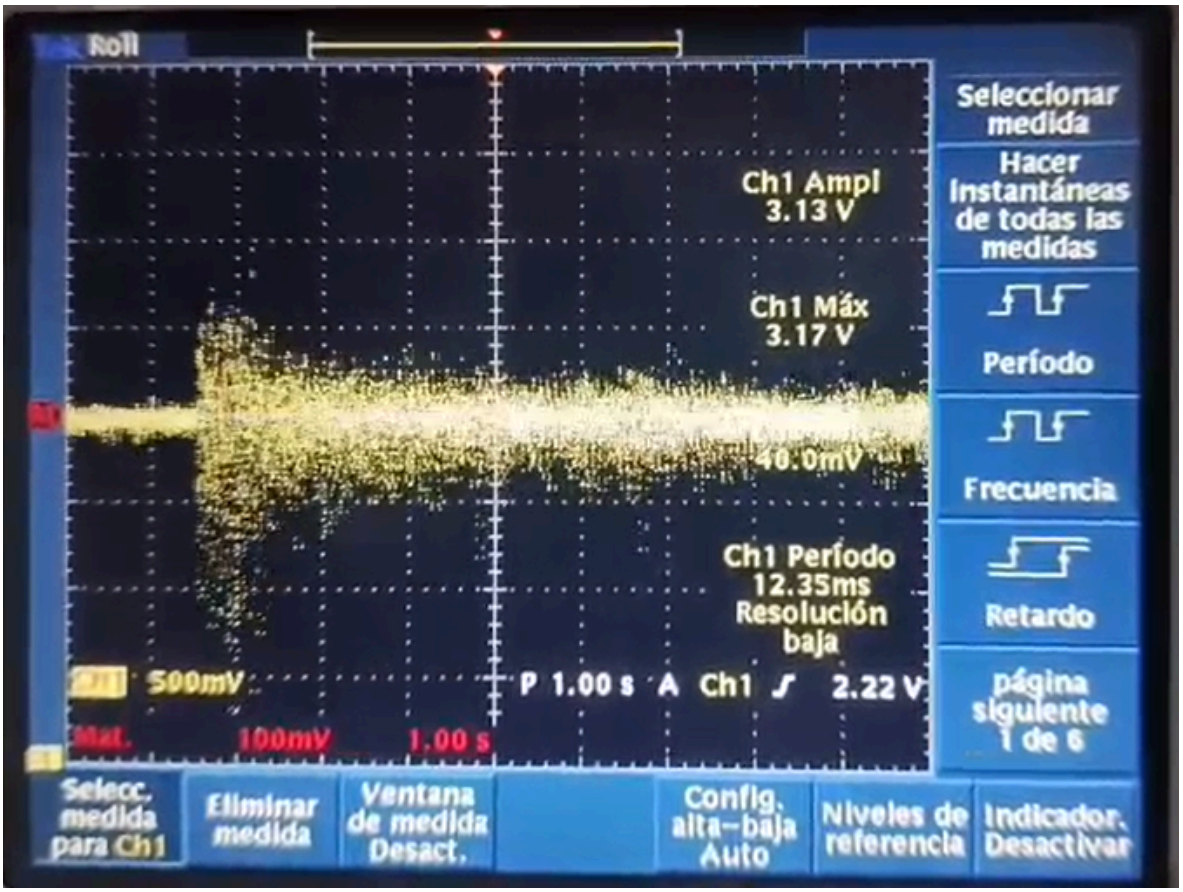


Figura 12. Registro de la señal sEMG reproducida mediante el DAC del ESP32 en GPIO25, con una base de tiempo de 1.00 s/div y una escala vertical de 500 mV/div. Se observa una tensión máxima de 3.17 V y una amplitud pico a pico de 3.13 V.

Cálculo de los valores teóricos

a) Periodo de repetición

$$T = \frac{N}{f_s} = \frac{30720}{2048} = 15\ s$$

b) Duración de un escalón

$$T_s = \frac{1}{2048} \times 1000 = 0.48828\ ms$$

c) Tensión máxima y mínima:

Considerando un DAC ideal de 8 bits, alimentado con 3.3 V, cuyo código digital varía entre 0 y 255:

$$V_{DAC} = \frac{D}{255} \times 3.3$$

$$V_{\max} = \frac{255}{255} \times 3.3 = 3.30\ V$$

$$V_{\min} = \frac{0}{255} \times 3.3 = 0\ V$$

Estos valores son ideales; la tensión real depende del comportamiento del DAC del ESP32.

Simulación escalonada de la salida del DAC a tensión

Se realizó una simulación de la salida del DAC mediante Google Colab, convirtiendo las 30 720 muestras digitales de 8 bits a valores de tensión comprendidos idealmente entre 0 y 3.3 V. La señal se representó durante un intervalo de 15 segundos y se amplió una sección para visualizar su comportamiento escalonado, con una duración teórica de 0.488 ms por muestra. Al comparar los resultados con el osciloscopio, se observaron tensiones máxima y mínima de 3.17 V y 0.040 V, respectivamente, próximas a los límites ideales del DAC.

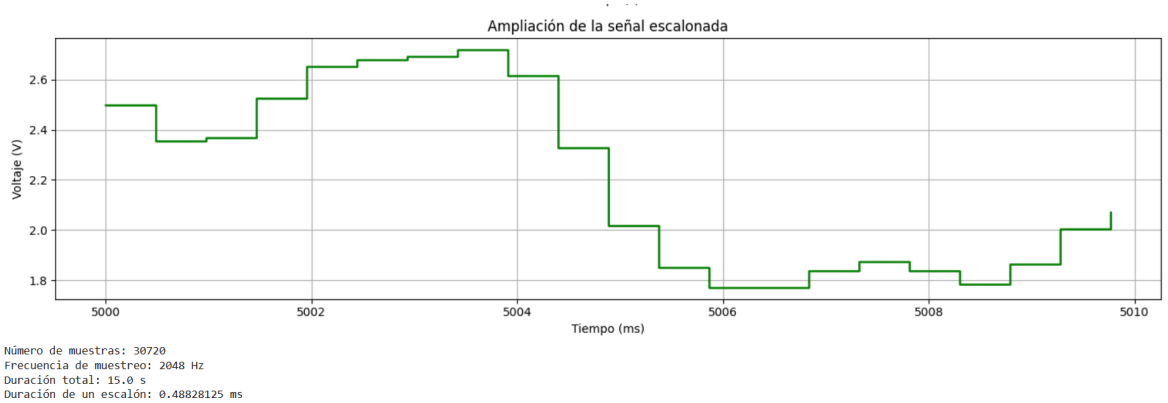


Figura 13. Simulación de la salida escalonada del DAC del ESP32 a una frecuencia de muestreo de 2048 Hz.

5. Resultados y discusión

- Comparar el periodo de repetición y la duración del escalón medidos con sus valores teóricos, y atribuir las diferencias a sus causas (resolución de la base de tiempo del osciloscopio, precisión de la temporización del firmware, entre otras).

El periodo de repetición de la secuencia presentó un valor teórico de 15 s, coincidiendo con los 15 s medidos experimentalmente. Por otro lado, la duración teórica de cada escalón fue de 0.488 ms; sin embargo, no se obtuvo un valor medido debido a las limitaciones de resolución de la base de tiempo del

osciloscopio, por lo que sería necesario ampliar la escala temporal y utilizar cursores. Aunque no se observaron diferencias en el periodo de repetición reportado, pueden existir pequeñas desviaciones debido a la precisión de la temporización del firmware, que utiliza intervalos de 488 µs en lugar de los 488.28 µs teóricos, así como a la resolución del osciloscopio y las condiciones de medición.

- A partir de las tensiones máxima y mínima medidas, estimar el rango real del DAC y la tensión correspondiente a un nivel de cuantización. Comparar con los valores nominales de la hoja de datos del ESP32.

  A partir de las mediciones obtenidas con el osciloscopio, se registró una tensión máxima de 3.17 V y una mínima de 0.04 V, obteniéndose un rango real de salida del DAC de 3.13 V. Considerando que el ESP32 posee un DAC de 8 bits, con 256 niveles de cuantización, se estimó una tensión de 12.27 mV por nivel (1 LSB), calculada mediante 3.13/255. Según la documentación de Espressif, el DAC permite generar tensiones aproximadamente entre 0 V y su voltaje de referencia, que idealmente corresponde a 3.3 V, con una resolución nominal de 12.94 mV por nivel. La diferencia entre ambos valores puede atribuirse a las características no ideales del DAC, las variaciones de alimentación y las condiciones de medición. Por lo tanto, el rango obtenido experimentalmente fue cercano al comportamiento ideal esperado del ESP32.
- Comparar las amplitudes relativas de los tres tramos observadas en el osciloscopio con las de las gráficas de la Etapa A, e indicar qué error en el escalado de la Etapa B habría alterado esa relación.

  Al comparar las gráficas de la Etapa A con la secuencia escalada de la Figura 8, se observó que el reposo presentó la menor amplitud (aproximadamente entre 165 y 185 niveles DAC), seguido de la extensión del índice, mientras que el cierre de mano alcanzó la mayor variación (0 a 255). Esta relación se espera también en la salida del osciloscopio. Asimismo, el nivel medio cercano a 175 equivale teóricamente a 2.26 V, próximo a los 2.22 V registrados mediante el cursor. Un error que habría alterado estas diferencias sería normalizar cada registro de forma independiente, utilizando sus propios valores mínimo y máximo, ya que esto ampliaría artificialmente la señal de reposo hasta ocupar todo el rango de 0 a 255. Sin embargo, se empleó un escalado global para toda la secuencia, conservando las amplitudes relativas de las tres condiciones musculares.

- Estimar cuántos niveles del DAC ocupa el tramo de reposo y discutir el efecto de la cuantización sobre la representación de señales de baja amplitud.

  El tramo de reposo ocupa aproximadamente 26 niveles de variación del DAC, según los valores mínimo y máximo de 158 y 184 obtenidos del archivo `semg.h`. Esto representa alrededor del 10.2 % del rango total de 255 intervalos del DAC de 8 bits. Considerando una resolución ideal de 12.94 mV por nivel, la señal de reposo presenta una variación aproximada de 0.34 V. Debido a su baja amplitud, la cuantización puede provocar la pérdida de pequeñas variaciones de la señal, ya que los cambios inferiores a un nivel de cuantización no se representan individualmente. Esto genera una salida escalonada y una mayor distorsión relativa en señales de baja amplitud, reduciendo la fidelidad de su representación respecto a las señales de mayor actividad muscular.

- Explicar el método de temporización implementado y qué efecto tendría sobre la señal reproducida un error de temporización acumulativo.

  El firmware genera la señal con una temporización por software basada en micros() y programación absoluta: el instante de la siguiente muestra se calcula sumando PERIODO_US al instante programado anterior, y no al instante actual. De este modo, el tiempo de ejecución del resto del bucle no se acumula y no existe deriva por retardo. Sin embargo, el periodo se calcula con división entera (1 000 000 / 2048 = 488.28 µs, truncado a 488 µs), lo que equivale a una frecuencia de 2049.18 Hz y a una secuencia de 14.991 s en lugar de 15 s, un error relativo de 0.057 %. Este es un error sistemático y acumulativo en la duración total, pero su efecto sobre la señal es una compresión temporal uniforme que escala todas las frecuencias por el mismo factor, por lo que no altera apreciablemente el contenido espectral de la sEMG.

Por otra parte, la actualización de la pantalla OLED se ejecuta en el mismo bucle y bloquea la emisión de muestras durante la transferencia I²C; el atraso se recupera con la programación absoluta, pero introduce jitter en la señal reproducida.

6. Cuestionario y preguntas de ampliación

- ¿Por qué se valida una cadena de acondicionamiento de bioseñales con un simulador antes de conectarla a un paciente?

  Porque entrega una señal conocida y repetible, de modo que se puede verificar ganancia, ancho de banda, ruido y rechazo de interferencias de la cadena de acondicionamiento sin exponer a una persona a un equipo no validado, y comparar versiones del circuito con la misma entrada.

- ¿En qué se diferencian la EMG de superficie y la EMG de aguja en amplitud y ancho de banda? ¿Qué implicaría usar un registro de aguja como fuente de este simulador?

  La sEMG tiene amplitudes de decenas de µV a pocos mV y un contenido útil de ≈ 20–500 Hz (Merletti & Farina, 2016). La EMG de aguja registra potenciales de unidades motoras individuales, de mayor amplitud y un ancho de banda considerablemente más alto, por encima de los 500 Hz, hasta varios kHz. Usar un registro de aguja en este simulador exigiría un fs mayor a 2048 Hz y una escala de amplitud distinta, y no representaría la señal de superficie.

- ¿Es suficiente la frecuencia de muestreo de 2048 Hz para representar el contenido espectral útil de la sEMG? Justificar con el teorema de muestreo.

  El teorema de Nyquist exige $f_s > 2 \cdot f_{max}$. Con $f_{max} \approx 500$ Hz, hace falta $f_s > 1000$ Hz, y 2048 Hz da un margen (frecuencia de Nyquist de 1024 Hz) en la práctica suficiente para el contenido útil de la sEMG. Se mantiene un margen para el filtro antialiasing en un sistema real.

- ¿Qué efecto tiene la salida escalonada del DAC sobre el contenido espectral de la señal reproducida, y qué etapa se agregaría para atenuarlo?

  La retención de orden cero equivale a multiplicar el espectro por una función sinc, lo que atenúa las frecuencias altas dentro de la banda (≈ −0.9 dB a 500 Hz) y deja imágenes espectrales alrededor de múltiplos de fs (2048 Hz, 4096 Hz…). Se agrega un filtro pasa-bajos de reconstrucción (por ejemplo, un Sallen-Key de segundo orden con fc ≈ 500 Hz) tras el DAC.

- La amplitud en GPIO25 es del orden de volts, mientras que una sEMG real es del orden de mV. Proponer y dimensionar la etapa necesaria para llevar la señal a su amplitud real.

  El DAC entrega ≈ 3.13 V pico a pico sobre un nivel DC de ≈ 2.2 V, mientras que la sEMG real es del orden de mV. Hace falta (a) eliminar el nivel DC con acoplamiento por capacitor (pasa-altos de fc $\ll$ 20 Hz) y (b) atenuar con un divisor. El factor es k = (rango original en mV) / 3130 mV. Si el rango de los registros es ≈ 1.5 mV (valor aproximado, tomado de la Figura 5), $k \approx 4.8 \times 10^{-4}$. Ejemplo: R1 = 100 kΩ y R2 = 47 Ω da $k \approx 4.7 \times 10^{-4}$. Se recomienda un buffer (seguidor) para fijar la impedancia de salida, y verificar el ruido del divisor.

- Comparar las características del DAC integrado del ESP32 (resolución, rango de salida) con las de un DAC externo de mayor resolución. Citar las hojas de datos.

  El ESP32 tiene un DAC de 8 bits (256 niveles, LSB ≈ 12.9 mV a 3.3 V) en GPIO25/26 (ESP-IDF Programming Guide). Un DAC externo de 12 bits como el MCP4725 (I²C) tendría 4096 niveles, LSB ≈ 0.8 mV a 3.3 V. A mayor resolución, mejor representación del reposo, a costa de un componente adicional y una interfaz I²C más lenta.

7. Conclusiones

- Se logró construir una secuencia de sEMG a partir de tres registros reales de GRABMyo, obteniéndose 30 720 muestras correspondientes a 15 segundos a una frecuencia nominal de 2048 Hz. El escalado global permitió conservar las diferencias de amplitud entre las condiciones musculares.

- La salida del DAC del ESP32 presentó una tensión máxima de 3.17 V y una mínima estimada de 0.04 V, obteniéndose un rango de 3.13 V y una resolución de cuantización estimada de 12.27 mV/LSB, valores próximos a los ideales para un DAC de 8 bits alimentado con 3.3 V.
- El cierre de mano presentó la mayor variación digital (255 LSB), seguido de la extensión del índice (94 LSB) y el reposo (26 LSB). Esto confirmó que el escalado global conservó las amplitudes relativas de los registros, aunque la cuantización limita la representación de pequeñas variaciones musculares.
- La temporización implementada mediante micros() permitió programar la reproducción continua de la secuencia. Sin embargo, el uso de un intervalo entero de 488 µs produce un periodo calculado de 14.991 s, ligeramente inferior a los 15 s nominales. La duración experimental de los escalones no pudo verificarse con la escala temporal de la captura disponible.
- El simulador desarrollado constituye una fuente reproducible de señales para futuras pruebas de instrumentación biomédica. No obstante, antes de utilizarlo para validar una cadena de acondicionamiento de sEMG, será necesario reducir su amplitud al orden de los milivoltios, eliminar la componente continua y considerar un filtrado de reconstrucción.

8. Materiales y código

Tabla 3. Lista de materiales y equipos utilizados.

Componente / equipo	Cantidad	Marca / modelo u observación
ESP32	1	WROOM/DevKit
Cable USB	1	Genérico
Lapto con acceso a internet, cuenta de Google (para Google Colab) y Arduino IDE con soporte para ESP32	1	Asus
Osciloscopio	1	Genérico
Puntas de osciloscopio	1	Genérico
Cables macho - hembra	2	Genérico

Tabla 4. Código fuente y enlace de repositorio.

Archivo / módulo	Descripción breve	Plataforma	Enlace al repositorio
Informe 3	Documentación usada para el informe, código del ESP32, semg.h y el informe mismo	Git Hub	https://github.com/RRORRIGO/Biomedical-Instrumentation
ESP32_sEMG	Simulación con pantalla OLED porque no integra osciloscopio	Wokwi	https://wokwi.com/projects/479559784319366145

Como evidencia complementaria, se conservó en Google Drive el video de la simulación del sistema, en el que se visualiza la reproducción de la secuencia completa correspondiente a las tres condiciones musculares: reposo, cierre de mano y extensión del índice. La duración nominal de la secuencia es de 15 segundos, distribuidos en tres intervalos de 5 segundos. Este registro audiovisual complementa las gráficas generadas en Google Colab y permite comprobar la continuidad de la reproducción simulada. Así como también se logra evidenciar la simulación en Wokwi porque no se encontró una librería capaz de funcionar como un ESP32 real, solo como representación visual. En Wokwi se presentan los 15 segundos que demora todo el proceso. Además adjuntamos el link en la tabla 4 donde se presenta todos los códigos y listo para simular.

Enlace a fotos y video adicionales del sistema en funcionamiento (implementación y simulación, demostración de los 15 segundo en woki y en osciloscopio)	LABORATORIO 3 INSTRUMENTACIÓN BIOM…

9. Referencias bibliográficas

Referencias de la guía:

- Jiang, N., Pradhan, A., & He, J. (2024). *Gesture Recognition and Biometrics ElectroMyogram (GRABMyo)* (versión 1.1.0). PhysioNet. https://doi.org/10.13026/89dm-f662
- Pradhan, A., He, J., & Jiang, N. (2022). Multi-day dataset of forearm and wrist electromyogram for hand gesture recognition and biometrics. *Scientific Data*, 9, 733. https://doi.org/10.1038/s41597-022-01836-y
- R. Merletti y D. Farina, Eds., Surface Electromyography: Physiology, Engineering, and Applications. Wiley-IEEE Press, 2016. https://doi.org/10.1002/9781119082934
- Espressif Systems, "Digital-to-Analog Converter (DAC)," ESP-IDF Programming Guide, documentación técnica del ESP32. https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/dac.html
- J. G. Webster, Ed., Medical Instrumentation: Application and Design, 4.ª ed. Wiley, 2010.
- IEC 60601-1, Medical electrical equipment – Part 1: General requirements for basic safety and essential performance.
- C. E. Shannon, "Communication in the presence of noise," Proc. IRE, vol. 37, n.° 1, pp. 10–21, 1949.
- A. V. Oppenheim y R. W. Schafer, Discrete-Time Signal Processing, 3.ª ed. Pearson, 2010.
- R. P. Sallen y E. L. Key, "A practical method of designing RC active filters," IRE Trans. Circuit Theory, vol. 2, n.° 1, pp. 74–85, 1955.